

➤ Array Basics :-

1. Create array.

marks = [0, 0, 0, 0, 0]

names = [null, null, null]

marks.length = 5.

2. initialize give values to array

a = {10, 20, 30, 40, 0, 0, 0, 0}

size = 4

a[2] = 30.

3. Insert element in middle.

Shift = 0.

Insert at (index 1, val 15) (a, 4, 1, 15) size increase by 1

→ i = 4, 4 > 1 → t a[4] = a[4-1] shift = 1 A[4] = 40

5 → i = 3, 3 > 1 → t a[3] = a[3-1] shift = 2 A[3] = 30

i = 2, 2 > 1 → t a[2] = a[2-1] shift = 3 A[2] = 20

i = 1, 1 > 1 - false stop shifting, A[1] = 15

a = [10, 15, 20, 30, 40, 0, 0, 0] inserted 15 shift = 3

size = 5

4. Insert 50 at end.

insertAt(a, size, 50, 5, 50)

Same loop

i = 5, 5 > 5 → false no shifting.

a[5] = 50, a = [10, 15, 20, 30, 40, 50, 0, 0]

size++ (size = 6).

$a = [10, 15, 20, 30, 40, 50, 0, 0]$

5. deleteAt (0, 6, 0)

$i = 0$ $i < \text{size} - 1$; $i++$ $a[i] = a[i+1]$

$i = 0$ $0 < 5 \rightarrow \checkmark$ $t = i = 0$ shift = 1 $a[5] = a[6]$

$i = 1$ $1 < 5 \checkmark$ shift = 2 $a[6] = a[7]$ $a[5] = 0$

$i = 2$ $2 < 5 \rightarrow \checkmark$ $a[7] = a[8]$ shift = 3

$i = 3$ $3 < 5 \checkmark$ $a[8] = a[9]$ shift = 4 $a[5] = 0$

$i = 4$ $4 < 5 \checkmark$ $a[9] = a[10]$ shift = 5

$i = 5$ $5 < 5$ false. $a = [15, 20, 30, 40, 50, 0, 0, 0]$
shift = 5

if $a[8] = 0$; give error / exception handle by try catch block
index 8 is invalid - exception (ArrayIndexOutOfBoundsException)

• Create jagged array // 2D array $t[i][j] \text{ for } i = \text{rows} - 1$
Size = 4 for r.

for $r = 0$; $r < 4$; $r++$

$r = 0$; $0 < 4 \checkmark$; $t[0] = t[1]$; $t[0][0] = t[0][0]$
 $t[0][1] = t[0][3]$

2) Array stacks.

It is generic type Array

$T[]$ data.

top = -1 — index.

define array stack with capacity = 4

1) Push: i) check $\text{isFull}()$ — when $\text{top} = \text{data.length} - 1$
 $\text{data}[++\text{top}] = \text{item}$.

Push 10. — $\text{top} = -1 = 3 \times$ — is not full

$++\text{top}$: $\text{top} = 0$ $0 = 3 \times$, push.

push 20. $\text{isFull} = 0 = 3 \times$ — not full

$\text{top} = 1$, $1 = 20$, push.

push 30. $\text{isFull} = 1 = 3 \times$ not full

$\text{top} = 2$, $2 = 30$ push.

2. Peek: check is Empty - if top == -1
stack = [30, 20, 10]
 ↑
 top

check is not Empty
return [top] - return 30

3) pop: check is isEmpty() - top == -1
stack = not empty (top=2), item = data[top] ~~pop~~
top-- , item = data[20]
return item top-- (top=1)
return = 20, 10

4) push again 40.
stack = [20, 10]
top=1, check is full = top = data.length-1
1 = 3 x not full
push(40) top=2, 2 = 40,

push 50, stack [40, 20, 10]
top=2, 2 = 3 x not full
push(50), top=3, 3 = 50,

push 60, stack [50, 40, 20, 10]
top=3, 3 = 3 ✓ is full

it give exception. "stack overflow - capacity" - 4

if it is not Empty then poped in LIFO order
from [50, 40, 20, 10] → stack underflow

3) balanced Brackets :-

String [] test = "{ [() ()] }"

create Deque = ArrayDeque for character
if check opening & closing brackets are correct.

if check if ch == '[' || ch == '(' || ch == '{'
then push in stack.

In this. First { push in stack - stack = [{,
then push [- stack = [{, []

then push (- stack = [{, [, (]

then push) but it not satisfy on +
'{', '[', '(' are push.

then go else if

if (ch == ')' || ch == '}' || ch == ']')

then check isEmpty - return false.

if (!matches (open, ch)

check if open == '(' & close == ')' then match.
Print balance.

- for this.

open = { & & ch = } op = [& & ch =] , open = (& & ch =)
satisfy

It is balanced.

for this = ([])

open (& & ch = [, open =) & & ch =]

not satisfy - not balanced.

Circular Queue:

Object [] data;

front = 0

size = 0

• create circular queue (int) = capacity = 5.

• isEmpty = return size == 0;

• isFull() - size == data.length

size()

1) push / enqueue - { 10, 20, 30, 40, 50 } for (int x = 0; x < data.length; x++)

enqueue. check isFull $\rightarrow$ ~~B~~ == ~~5~~ $\times$ not full

rear = (front + size) % data.length, $(0 + 0) \% 5 = 0$

data[rear] = item 0 = 10

size++ size = 1

• rear = $0 + 1 \% 5 = 1$.

data[1] = 20. size = 2

• rear = $0 + 2 \% 5 = 2$.

data[2] = 30 size = 3

• rear = $0 + 3 \% 5 = 3$

data[3] = 40 size = 4

• rear = $0 + 4 \% 5 = 4$ size = 5

data[4] = 50

• size == data.length, $5 == 5$ true it is
isFull.

2) dequeue. $\rightarrow$ dequeue()

• isEmpty = size == 0

Size = 5, size == 0 false is not Empty

queue = [10, 20, 30, 40, 50]

dequeue - 1 item = (1) data[front]

data[front] = null;

front = (front + 1) % data.length;

See --;

return item;

return [0, 20, 30, 40, 50]
F R.

front = 0

data[0] = 10

data[0] = null

front = 0 + 1 % 5 = 1

Size = 4

front = 1, data[1] = 20, data[1] = null

front = 1 + 1 % 5 = 2, size = 3

[0, 0, 30, 40, 50]
F R

front = 2, data[2] = 30, data[2] = null

front = 2 + 1 % 5 = 3, size = 2

[0, 0, 0, 40, 50]
F R

Power only x 3

front = 3, data[3] = 40, data[3] = null

front = 3 + 1 % 5 = 4, size = 1

[0, 0, 0, 0, 50]
F R

front = 4, data[4] = 50, data[4] = null

front = 4 + 1 % 5 = 5, size = 0

[0, 0, 0, 0, 0]

Size = 0 so, it is Empty - Queue underflow

Size = 2, front

enqueue. 60, 70, 80.

front = 3, data[3] = 2, size = 2.

front = rear = 3 + 2 % 5 = 0.

data[0] = 60.

[60, 0, 0, 40, 50]
R F F

Size = 3

70, 80 like this.

70. front = 3, size = 3

$$\text{rear} = \text{front} + \text{size} \times .5 \\ = 3 + 3 \times .5 = 4$$

data[0] = 70. size = 4

[60, 70_P, 0, 40_F, 50]

80. front = 3, size = 4 data[2] = null

$$\text{rear} = 3 + 4 \times .5 = 2 \quad \text{data}[2] = 80$$

size = 5.

[60, 70, 80_R, 40_F, 50]

- When we want to push 90 or enqueue 90
size == 5. 5 == 5 is full to
queue is overflow.

Dynamic Array:

create DynamicArray of capacity 4.

size = 0, copies = 0

array = [0, 1, 2, 3]

i = 1, list.add(i * 10) = add(10)

for add: check size == data.length grow()

data[size++] = item

0 == 4 x not true.

data[0] = 10. size = 1

size = 1, capacity = 4 add(20)

1 == 4 x false.

data[1] = 20. size = 2

add(30), size = 2, capacity = 4

2 == 4 false. data[2] = 30, size = 3

add(40) , size = 3 , capacity = 4
 3 == 4 false
 data[3] = 40 size = 4

• add(50), size=4, capacity=4
~~size~~ 4 true. it is grow()

```
int oddcup = data.length;
```

```
int oldcap = data.length;
newcap = Math.max (old + (old > 7), old + 1);
```

old = 4, u777 = 2

old + old 777

$$4 + 2 = 6$$
$$\text{old } \text{cnp} + 1 = 4 + 1 = 5$$
$$\text{Math: } \max(6, 5) = 6$$

Capacity grow to 6

and we can copy array and add more ele.
until size = 20.

• xmove(0):

remove(0):
old = get(cid * 4) = 10

idx = 0, size = 20

```

idx = 0, size = 20
arrayCopy(data, idx + 1, data, idx, size - idx - 1) size--;

```

data[19] = null

```
return [ 20, 30, 40, 50, ... ]  xmax = 110
```

size = 19.